

In Angular, a template is a chunk of HTML. Within a template, you can use special syntax to leverage many of Angular's features

Reactive forms:

reactive form Date: / / also Page no: /

- In a model-driven approach, the model which is created in the .ts file is responsible for handling all the user interactions/validations. For this, first we need to create the model using Angular's inbuilt classes like `FormGroup` and `FormControl` and then, we need to bind that model to the HTML form.
 - This approach uses the Reactive forms for developing the forms which favors the explicit management of data b/w the UI and the Model. With this Approach, we create the tree of Angular form controls and bind them in the native form controls. As we can create the form-control directly in the component, it makes it a bit easier to push the data between the data models and the UI elements.
 - model driven forms are more powerful and more flexible.
 - There are some things that we can't do with template driven form but with model driven form we can perform those things. Like if we want to detect if any changes occur in any variable we can do it with model driven form. we can easily do two-way data binding with model driven form in large application.
- ✓ Reactive form in Angular is a technique to manage your form in a reactive manner. it means that you can manage your form and validation from our component itself.
- ✓ These forms are the best option when we have a complex form requirement.
- ✓ Compared to Template driven forms, reactive form are more suitable bcoz we can define validations and model form from component, that gives us more control on form.

Date: / / Page no:

* Primarily it is also called "Model Driven Forms", because model acts as a mediator b/w component and template.

- It's really easy to implement, like using model in controller with form control object and binding it to component form control template.

Prerequisite

- We need to import `ReactiveFormsModule` in our `app.module.ts` file.

Reactive Forms Features.

- More flexible, but needs a lot of practice.
- Handles any complex scenarios.
- More component code and less HTML markup.
- No data binding is done (immutable data model preferred by most developers).
- Reactive transformations can be made possible such as
 - Handling an event based on a debounce time.
 - Handling events when the components are distinct until changed.
 - Adding elements dynamically.
- Easier unit testing.

* FormControl

- It tracks the value of the controls and validates the individual control in the form.
- In Reactive Forms we initialize `FormControl` object to use form functionality into our component, which engaged with our HTML form.

First name required

- And when we update our form controls value then it will directly be reflected to our FormControl object that we created previously.

- FormGroup
- Tracks the validity and state of the group of FormControl instance or moreover, we can say the FormGroup to be a collection of FormControl.
- like:- validations, name of input control etc.

FormBuilder

This helps us to develop the forms along with their initial value and these validation.

Reactive forms with validation.

- The most important part of any form is to add the validations in the application.
- When we do the `fname: new FormControl()`, the first argument in the FormControl is the initial value and the second value is the validations that we can add these
- The only change in the component would like below

```
* this SignupForm = FormBuilder.group({  
  fname: ['', validators.compose([validators.required, validators.maxLength(15), validators.minLength(1)])],  
  lname: ['', [validators.required, validators.maxLength(19)]],  
  emailid: ['', [validators.required, validators.email]],  
  userpassword: ['', validators.required]
```


FormGroup get value and reset()

- To get the value of form control named as name after form submit, we need to write the code as below.

```
this.userService.get('name').value
```

- if we want to get all values of the form then we can write codes as given below

```
this.userService.value
```

- To reset the form we need to call reset() method on FormGroup instance. Suppose userService is the instance of FormGroup then we create a method as given below

```
resetForm() {
```

```
  this.userService.reset();
```

```
}
```

- We can ~~add~~ call the above method using a button as follows

```
<button type="button" (click)="resetForm()">Reset </button>
```

If we want to reset form with some default values, then assign the default values with form control name of the form.

```
resetForm() {
```

```
  userService.reset({
```

```
    name: 'Mahesh';
```

```
    age: 20
```

```
  });
```


FormGroup setValue() and PatchValue()

Date: / / Page no:

- Angular FormGroup has methods such as `setValue()` and `patchValue()`. Both sets the value in form control of FormGroup. `setValue()` sets the value in each and every form control of FormGroup. We cannot omit any form control in `setValue()` but when we want to assign only few form controls of FormGroup then we need to use `patchValue()`. Both methods are used in the same way.
- * It is necessary to mention all form controls in `setValue()` otherwise it will throw error.
- * When we use `patchValue()` then it is not necessary to mention all form controls.

valueChanges() and statusChanges()

- `valueChanges` and `statusChanges` are properties of `FormControl`, `FormArray` and `FormGroup` classes. `valueChanges` and `statusChanges` both return Observable instance and we can subscribe them to get data.
- `valueChanges` is a property of `AbstractControl` that emits an event every time when the value of control changes either using UI or programmatically. `valueChanges` property is available in `FormControl`, `FormArray` and `FormGroup` classes because they inherit `AbstractControl` class.
- `statusChanges()`

- An Angular APP is defined by a set of Ngmodules

- StatusChanges is a property of AbstractControl that emits an event every time when the validation status of the control is recalculated. StatusChanges property is available in FormControl, FormArray and FormGroup classes because they inherit AbstractControl class.

FormArray in Angular.

- It is a class that tracks the value and validity state of array of FormControl, FormGroup and FormArray. A FormArray aggregates the values of each child FormControl into an array. It calculates its status by reducing the statuses of its children. For example, if one of the controls in a FormArray is invalid, the entire array becomes invalid.
- npm package @angular/forms.
- Module import { FormArray } from '@angular/forms';

Nested FormArray

- Nested form arrays can come in handy when you have a group of form controls but you're not sure how many there will be. FormArray allows you to create new form controls dynamically.
- Access the array: you can access the associated FormArray using the get method on the parent FormGroup

1. Angular distinguishes components from services to increase modularity and reusability.
2. Angular is written in typescript Date: / / Page no:

Ex: `this.form.get('cities').`

- * **Get the value:** the value property is always synced and available on the `formArray`.
- * **Set the value:** you can set an initial value for each child control when instantiating the `FormArray`, or you can set the value programmatically later using the `setValue` and `patchValue` methods.

- **Add new controls:** you can add new controls to the `FormArray` dynamically by calling its `push` method

Ex: `this.form.get('cities').push(new FormControl());`

Session management

`ngOnInit() {`

`localStorage.setItem('localData', 'localStorage');`

`sessionStorage.setItem('sessionData', 'SessionStorage');`

`console.log(localStorage.getItem('localData'));`

`console.log(local sessionStorage.getItem('sessionData'));`

3

- **Deleting entries**

Deleting an entry with the `removeItem` method.

`localStorage.removeItem('key');`

- `localStorage.clear();` → clears everything that's stored in local storage.

Angular Module Loading: Eager, Lazy and Preloading

- An effective loading strategy is key to a successful single-page application.
- A module can be loaded eagerly, lazily and preloaded.
- Eager loading: In an eager loading scenario, all of the modules and functions are loaded on application startup. The root module is always eagerly loaded and in some cases you will use eager loading for additional features.
- Lazy loading: It is loading modules on demand.
- Preloading: It is loading modules in background just after application starts.

Eager loading

When you're working with Angular routes, in the beginning, all of your loading is eager. Bcoz wlo doing anything but including routes in your app, they will default to loading this way. It means everything loads all at once. If you had an app with three sections - home, admin, dashboard - all three would load when the user visits the browser.

- Eager loading means the slowest load times for this reason. When you have a small app this is okay bcoz the total size will not be large enough to see a benefit from the other two loading strategies.

typescript 5.2.1 time में है Compile time में

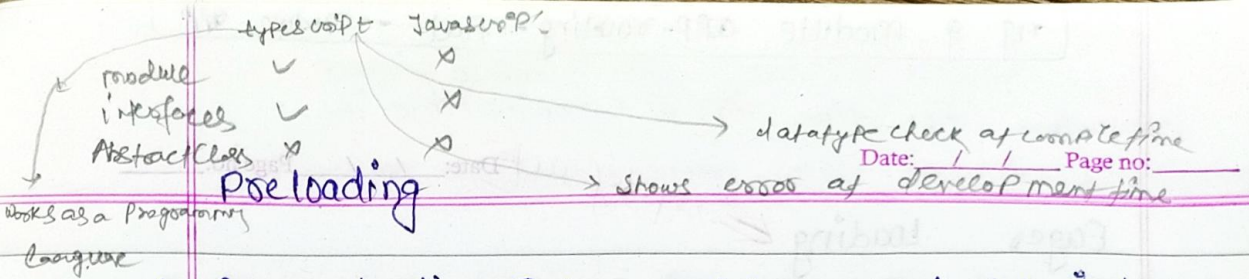
Eager Loading

by default

1. In eager loading module, Feature modules are loaded before application start on first hit. To load a Feature module eagerly, we need to import that module in application module
ie AppModule using imports metadata of @NgModule decorator.
2. When a module is loaded, it loads all the imported modules, Configured Components, Services, Custom pipes etc
3. Modules are loaded in the order they are configured in imports metadata.
4. Eager loading is good for small applications bcoz at the first hit of the application all the modules are loaded and all the required dependencies are resolved. Now the access to the application will be faster.

Lazy Loading

```
Const routes : Routes = [  
  { Path: 'Country', loadChildren: 'app/country/country.module#CountryModule' },  
]
```

Preloading

1. In preloading, feature modules are loaded in background asynchronously. In preloading, modules start loading just after application starts.
2. When we hit the application, first AppModule and modules imported by it, will be loaded eagerly. Just after that modules configured for preloading is loaded asynchronously.
3. preloading is useful to load those features which are in high probability to be visited by users just after loading the application.
4. To configure preloading, angular provides PreloadingStrategy property which is used with RouterModule. forRoot in routing module.
5. To configure preloading feature modules, first we will configure them for lazy loading and then using Angular in-built PreloadAllModules strategy, we enable to load all lazy loading into preloading modules.
6. Using PreloadAllModules Strategy, all modules configured by loadChildren property will be preloaded. The modules configured by loadChildren property will be either lazily loaded or preloaded but not both. To preload only selective modules, we need to use custom preloading strategy.

routerLink = "Person" ✓

routerLink = "/person" X

= ". / Person" ✓

@ViewChild()

Date: / / Page no:

When building an application with Angular you may come across the following annotations

- @ViewChild
- @ViewChilden
- @ContentChild
- @ContentChildren

ViewChild()

- `ViewChild` are used for Component Communication in Angular. Therefore, if a Parent Component wants access of child component then it uses `ViewChild`. Any Component, directive, or element which is part of a template is `ViewChild`.
- `@ViewChild()` decorator can be used to get the first element or the directive matching the Selector from the view Dom. `@ViewChild()` provides the instance of another Component or directive in a Parent Component and then Parent Component can access the methods and properties of that Component or directive. In a Parent Component we can use `@ViewChild()` for Components, directives and template reference variable with `ElementRef` or `TemplateRef`. To use `@ViewChild()` we need to pass child Component name or directive name or template variable as an argument.
- if you want to access following inside the Parent Component, use `@ViewChild` decorator of Angular
 1. Child Component
 2. Directive
 3. Dom Element.

@ViewChild() using Template Variable

Date: / / Page no:

- * @ViewChild() can instantiate ElementRef corresponding to a given template reference variable. The template variable name will be passed in @ViewChild() as an argument. In this way Component will be able to change the appearance and behavior of element of given template variable.
- @ViewChild() provides the instance of another Component or directive in a parent Component and then parent Component can access the methods and properties of that Component or directive.
- viewChild are used for components communication in Angular. Therefore, if a parent Component wants access of child Component then it uses viewChild.

<ng-template>

ng-template is 37-22 * 7 m 11/11

Date: / / Page no:

- <ng-template> is an angular element for rendering HTML. it is never displayed directly. it can be displayed using structural directive that we use all the time: ngIf, ngFor and ngSwitch.
- Before rendering HTML, angular replaces <ng-template> with a comment. <ng-template> can be used with structural directive, viewContainersRef etc.
- if you put some HTML inside of an <ng-template> tag, it not only won't be on the screen, but it won't be in the DOM either. Angular will replace the <ng-template> tag and its contents with a comment. The key is that <ng-template> will only be displayed if used in partnership with a structural directive.
- A rendered <ng-template> doesn't itself get turned into a DOM element, only the contents are rendered to the DOM.
- <ng-template> is a virtual element and its contents are displayed only when needed (based on conditions)
- <ng-template> should be used along with structural directives like [ngIf], [ngFor], [ngSwitch] or custom structural directives.
- <ng-template> never meant to be used like other HTML elements. it's an internal implementation of Angular's structural directives
- when you use a structural directive in angular we will add a prefix asterisk (*) before the directive name. This asterisk is short hand notation for <ng-template>
- Whenever Angular encounter with a asterisk (*) symbol, we are informing Angular saying that it is a structural directive and angular will correct directive

Attribute to `<ng-template>` element.

- `<ng-template>` is not exactly a true web element when we compile our code, we will not see a `<ng-template>` tag in HTML DOM.
- Angular will evaluate the `<ng-template>` element to convert it into a comment section in HTML DOM.

`<ng-container>` → *ଏହା ଏକ logical container (ng-template ମିଳିବା ନାହିଁ) ଏବଂ condition ଥିବା ଶୁଦ୍ଧି ନାହିଁ)*

- * `ng-container` is an element that's available in Angular 2+
- `<ng-container>` is a logical container that can be used to group nodes but is not rendered in the DOM tree as a node. `<ng-container>` is rendered as an HTML comment.
- In order to avoid having to create that extra `div`, we can instead use `ng-container` directive.
- `<ng-container>` is an Angular grouping element that is similar to `<ng-template>` in that it doesn't represent a DOM element. The difference is that it will always be rendered, whereas an `<ng-template>` will only be rendered if it is explicitly requested. `<ng-container>` are useful anywhere you need an extra container for some template elements, but don't want to (or can't) create a container such as a `div` to hold them with due to syntax.

For example: It is not allowed in Angular to put two structural directives on the same element. If you needed to loop through an array and display a `<td>` for each element, but only if a different condition was met, you may want to put both an `*ngfor` and `*ngIf` on the `<td>` element. Angular does not allow this, however, and wrapping the `<td>` in a `<div>` to hold one of the structural

directives is not valid HTML. The utility of `<ng-container>` shines here, where we can use the `<ng-container>` to hold a structural directive and contain the `<tr>` `<td>` w/o breaking the HTML layout.

HostListener()

`@HostListener()` function decorator allow us to handle event of the host element in the directive class.

`@HostListeners` lets us to listen for event on host element or component.

Route Guards in Angular

Date: / / Page no:

- Being web application developers, we are aware how a server checks the permissions for the user on the navigation and returns the result telling whether a user can access the resource or not. In the same way, we need to achieve this on the client-side. Route Guards are the solutions for that. We can use the route guards to control whether the user can navigate the route or not.

1. CanActivate

This helps to decide whether the route can be activated or not. It is used to check to see if a user can visit a route.

2. CanActivateChild

This helps to decide whether child routes can be activated or not. It is used to check to see if a user can visit a route's child.

3. CanDeactivate

This helps to decide whether the route can be deactivated or not. Generally, this is used to warn a user if they are navigating from the component. It is used to check to see if a user can exit a route.

4. Resolve

This performs route data retrieval before any activation of the route. It is used to perform route data retrieval before route activation.

5. CanLoad

Checks to see if the user can navigate to the module which is lazily loaded.

Each route in this array is a JavaScript object that contains two properties, first property is Path, Second property is component.

→ `const routes: Routes = [ path: ' ', component: ' ' ]`. Page no: _____

Route Guards in Angular

- Before starting to implement the guards, let's have some basic information about route guards.
 - Guards are always implemented as service that needs to be provided so we always make them `@Injectable` while creation.
 - Guards always return true or false telling you if access is allowed or not.
 - Guards can also return the observables or promises which can resolve into the Boolean value finally and will be returned.
 - Every Route guard must be imported to the application or Root module.
 - As the Route guards are services they must be registered in the provided Application module.
 - Generally, in our application, we have a need where we want to check if the user is logged in or not. In that case, we can decide if we want to activate that route for that user or not.
 - Angular provides `CanActivate` Route for that.
- ng g g guardname.

Session Management

cookies (5ms)

Date: / /

Page no: _____

- Storing JSON Objects.
- Only strings can be stored with localStorage or Sessions - storage, but you can use JSON.stringify to store more complex objects and JSON.parse to read them.

// create them

```
let myObj = { name: 'Skip', breed: 'Labrador' };  
localStorage.setItem(key, JSON.stringify(myObj));
```

// Read them

```
let item = JSON.parse(localStorage.getItem(key));
```

Checking for items.

Here's how you can test for the presence of items in the localStorage:

```
if (localStorage.length > 0)
```

```
{
```

```
  // we have items
```

```
} else
```

```
{
```

```
  // No items
```

```
}
```

Checking for Support.

Test for localStorage support by checking if it's available on the window object.

```
if (window.localStorage)
```

```
{ // localStorage supported }
```


- The Safe Navigation Operator is also known as the "Elvis Operator". This operator is very useful to protect against null and undefined values in property paths.

Date: / / Page no:

- * Angular templates are dynamic. When Angular renders them it transforms the DOM according to the instructions given by Directives. A directive is a class with a `@Directive()` Decorator.
- * A Component is technically a directive. However, components are so distinctive and central to Angular applications that Angular defines the `@Component()` decorator, which extends the ~~@Decorator~~ `@Directive()` decorator with template-oriented features.

Template reference variables (#var)

A template reference variable is often a reference to a DOM element within a template. It can also refer to a Directive (which contains a Component), an element, `TemplateRef` or a web component.

Use the `(#)` to declare a reference variable. The following reference variable, `#phone`, declares a phone variable on an `<input>` element.

```
<input #phone placeholder="Phone numbers" />
```

You can refer to a template reference variable anywhere in the Component's template.

Angular assigns each template reference variable a value based on where you declare the variable.

- (i) If you declare the variable on a Component, the variable refers to the component instance.

You can use the ref - Prefix alternative to #.

ref - Phone

Date: / / Page no:

- (ii) If you declare the variable on a Standard HTML tag, the variable refers to the element.
- (iii) If you declare the variable on an `<ng-template>` element, the variable refers to a `TemplateRef` instance, which represents the template.
- (iv) If the variable specifies a name on the right-hand side, such as `#var = "ngmodel"`, the variable refers to the directive or component on the element with a matching `exportAs` name.

Form (Reactive form) - it's available when you are binding to a form.

`FormControlName` → to bind individual control

`FormGroup` → to bind form

`FormGroupName` → to bind nested form.

`FormControl` →

`<option selected [ngvalue] = "null"> Choose -- </option>`

↑
null value
↓
Choose --

`reactiveForm.controls.address.controls.zipCode` ← not looking in Reactive form

↓
return the control of abstract control class
↓
this is present inside form group class.

↑ so create get method → `get()`.
↑ both are complex
↑ like get zip code

let form = new FormGroup({
 address: new FormGroup({
 zipCode: new FormControl('')
 })
});
form.get('address').get('zipCode')

validators: Compose ([validators: required, validators: minLength(10), validators: ...])

When you apply multiple validators

Validates

Date: / / Page no:

Control.root → return root element of the complete form group tree to which this particular ~~form~~ ^{Control} belong.

↓
in this example root element will be the reactiveform

const group =

<formgroup>

Control.root.get('address');

to get the address form group which is nested inside the form group

Angular by default return Abstract

So cast to formgroup - control

group → is only exist if group is completed.

Nested Form

contains of the reactive form

⇒ A form that is embedded inside another form.

this.nestedForm = this.fb.group({

firstName: [], lastName: []

initiate the nested form

address: this.fb.array([])

Call functional group

this.addresses = this.fb.array([])

Template Driven form.

ngModel: To make our individual input field exposed to our typescript file so that we can extract the value, provide the validation etc.

ngModelGroup: to group up certain set of fields or control under ~~the~~ a single link.

ngForm: It is a ~~directive~~ ^{which is used} directive [↑] to a complex set of template driven form in our web application.

use NgStyle to set many inline styles simultaneously and dynamically, based on the state of component

Built-in attributes directives

Date: _____ Page no: _____

- NgClass - adds and removes a set of CSS classes
- NgStyle - adds and removes a set of HTML styles
- NgModel - adds two-way data binding to an HTML form element.

Note: To add or remove a single class, use class binding rather than NgClass

* Setting a single style value with style binding without NgStyle

```
<div [style.font-size] = "isSpecial ? 'x-large': 'smaller'">
```

This div is x-large or smaller

```
</div>
```

Show/hide vs NgIf

Hiding an element is different from removing it with NgIf.

When you hide an element, that element and all of its descendants remain in the DOM. All components for those elements stay in memory and Angular may continue to check for changes. You could be holding onto considerable computing resources and degrading performance unnecessarily.

NgIf works differently. When NgIf is false, Angular removes the element and its descendants from the DOM. It destroys these components, freeing up resources, which results in better user experience.

RxJS Library. (Reactive Extensions for JavaScript)

- Reactive Programming is an asynchronous programming paradigm concerned with data streams and the propagation of change.
- It is a library for reactive programming using observables that makes it easier to compose asynchronous or callback-based code.

Operators

Operators are functions that build on the observables foundation to enable sophisticated manipulation of collections.

For example, RxJS defines operators such as `map()`, `filter()`, `concat()` and `flatMap()`.

eg.

```
import { map } from 'rxjs/operators';
```

```
const nums = of(1, 2, 3);
```

```
const squareValues = map((val: number) => val * val);
```

```
const squaredNums = squareValues(nums);
```

```
squaredNums.subscribe(x => console.log(x));
```

```
// logs  
1  
4  
9
```

you can use pipes to link operators together. Pipes let you combine multiple functions into a single function. The `pipe()` function takes as its arguments the functions you want to combine, and returns a new function that, when executed, runs the composed functions in sequence.

A set of operators applied to an observable is a recipe—that is, a set of instructions for producing the values you're interested in. By itself, the recipe doesn't do anything. You need to call `subscribe()` to produce a result through the recipe.

* We use a Map with a pipe, which allows us to chain multiple operators together.

Date: / / Page no:

Reactive Programming

The reactive programming is all about creating the stream, emitting the value, error, or complete signals, manipulate, transfers or do something useful with the data streams.

* The observable invokes the next() callback whenever the value arrives in the stream. It passes the value as the argument to the next callback.

- Observers (Subscribers) registers three callbacks with the observable at the time of subscribing, i.e. next(), error() & complete().
- All three callbacks are optional.
- The observer receives the data from the observer via the next() callback.

→ takes only one argument, that can be iterated and converts it into an observable.

These are many ways to create observable in Angular. You can make use of Observable Constructors, and there are a no. of functions that are available which you can use to create new observables. These operators help us to create observable from an array, string, promise, any iterable etc. Here are some of the operators

- create
- defer
- empty
- from
- fromEvent
- interval
- of
- range
- throw
- timer

→ it converts almost anything that can be iterated to an observable

It is the method of observable object
It calls the observable constructor behind the scene

Anything that can be iterated can be converted to observable

Date: / / Page no:

Of vs from

Of

1. Accepts variable no. of arguments
2. emits each arguments as it is without changing anything

From

1. Accepts only one argument
2. Iterates over the argument and emits each value